

Funciones y programación funcional

JavaScript es un lenguaje funcional: las funciones son valores «de primera clase», igual que los números o las cadenas.

Índice

- 3.1 Declaración de funciones
- 3.2 Funciones de orden superior
- 3.3 Literales de función
- 3.4 Funciones flecha
- 3.5 Procesamiento funcional de matrices
- 3.6 Modo estricto
- 3.7 Comprobación de tipos de argumentos
- 3.8 Más o menos argumentos de los esperados
- 3.9 Argumentos predeterminados
- 3.10 Parámetros restantes y operador de propagación
- 3.11 Hoisting
- 3.12 Lanzamiento de excepciones
- 3.13 Captura de excepciones
- 3.14 La cláusula finally

INTRODUCCIÓN

Las funciones son valores «de primera clase», igual que los números o las cadenas

Se pueden guardar en variables, pasar como argumentos y devolver como resultado de otras funciones. Dominar este estilo es esencial para trabajar con JavaScript moderno.

3.1 DECLARACIÓN DE FUNCIONES

Declarar y llamar una función

- ✓ El nombre de la función
- ✓ Los nombres de los parámetros
- ✓ El cuerpo, que calcula y devuelve el resultado

💡 No se indican los tipos de los parámetros ni del resultado: **promedio('6','7')** concatena y da 33.5, no un error.

```
function promedio(x, y) {  
    return (x + y) / 2  
}  
  
let resultado = promedio(6, 7)  
// resultado es 6.5
```

3.1 DECLARACIÓN DE FUNCIONES

return sale de inmediato

```
function indiceDe(vector, valor) {  
  for (let i in vector) {  
    if (vector[i] === valor) return i  
  }  
  return -1  
}
```

⚠ Sin instrucción **return**, o con **return** sin expresión, la función devuelve **undefined**.

💡 Si a veces devuelves resultado y otras no, sé explícito: **return undefined**.

3.2 FUNCIONES DE ORDEN SUPERIOR

Las funciones son datos

Una función se puede **guardar en una variable**, pasar como argumento o devolver como resultado de otra función.

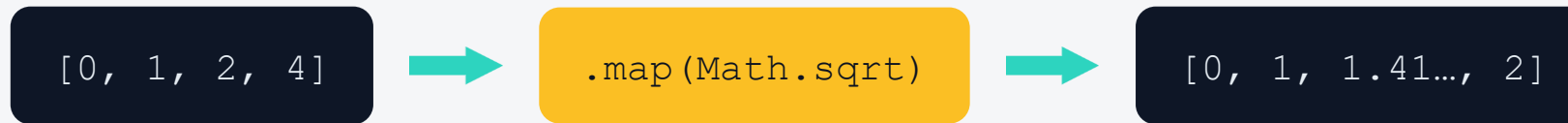
⚡ Una **función de orden superior** es una función que consume o produce otra función.

```
let f = promedio
let resultado = f(6, 7)
// se invoca la función guardada en f

f = Math.max
resultado = f(6, 7)
// ahora resultado es 7
```

3.2 FUNCIONES DE ORDEN SUPERIOR

Pasar una función como argumento



El método **map** aplica la función a cada elemento y devuelve un nuevo vector, sin modificar el original: se le llama función de orden superior.

```
vector.map(algunaFuncion) // forma general
```

3.3 LITERALES DE FUNCIÓN

Funciones sin nombre, para usar una sola vez

```
// con una función con nombre, declarada aparte
function multiplicarPorDiez(x) { return x * 10 }
resultado = [0, 1, 2, 4].map(multiplicarPorDiez)
```

```
// mejor: un literal de función anónimo, en línea
resultado = [0, 1, 2, 4].map(function (x) { return 10 * x })

const promedio = function (x, y) { return (x + y) / 2 }
```

💡 Piensa en el literal de función anónimo como el caso «normal»: una función con nombre solo le añade una variable que la guarda.

3.4 FUNCIONES FLECHA

Una sintaxis más concisa: =>

```
const promedio = (x, y) => (x + y) / 2
const multiplicarPorDiez = x => x * 10
const tirarDado = () => Math.trunc(Math.random() * 6) + 1
```

Parámetros

A la izquierda de la flecha. Un único parámetro no necesita paréntesis; sin parámetros, usa ().

Valor de retorno

A la derecha de la flecha. El token => debe ir en la misma línea que los parámetros.

3.4 FUNCIONES FLECHA

Cuerpos de bloque y objetos literales

```
const indiceDe = (vector, valor) => {  
  for (let i in vector) {  
    if (vector[i] === valor) return i  
  }  
  return -1  
}  
  
// con bloque, hace falta return
```

```
const estadisticas = (x, y) => ({  
  promedio: (x + y) / 2,  
  distancia: Math.abs(x - y)  
})
```

⚠ Si la flecha solo devuelve un objeto literal, envuélvelo entre paréntesis: si no, las llaves se leen como un bloque.

3.5 PROCESAMIENTO FUNCIONAL DE MATRICES

forEach, map y filter

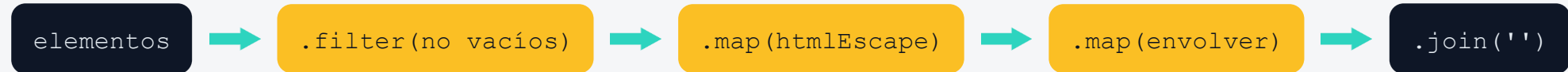
```
vector.forEach((elemento, indice) => { console.log(`${indice}: ${elemento}`) })
```

Método	Qué hace	Qué devuelve
forEach	Ejecuta una función por cada elemento	undefined (efectos secundarios)
map	Transforma cada elemento	Un nuevo vector transformado
filter	Selecciona según un predicado	Un nuevo vector filtrado

Es mejor evitar efectos secundarios y encadenar **filter** y **map** para transformar matrices en la forma deseada.

3.5 PROCESAMIENTO FUNCIONAL DE MATRICES

Un ejemplo: construir una lista HTML



Un estilo «qué, no cómo»: quitar cadenas vacías, escapar el HTML, envolver cada elemento en `<li>` y unirlo todo en una sola cadena.

```
const envolver = (etiqueta, contenido) => `<${etiqueta}>${contenido}</${etiqueta}>`

const lista = envolver('ul', elementos
  .filter(i => i.trim() !== '').map(escaparHtml).map(i => envolver('li', i)).join(''))
```

3.6 MODO ESTRICTO

Usa siempre modo estricto

`'use strict'` — primera línea del archivo, antes de cualquier código

- ✓ Asignar a una variable no declarada es un error: usa siempre `let`, `const` o `var`
- ✓ No se puede reasignar una propiedad global de solo lectura (`NaN`, `undefined`)
- ✓ Las funciones solo se declaran en el nivel superior de un script o función
- ✓ No se permiten parámetros duplicados en una misma función
- ✓ No se permiten literales octales con el prefijo `0`
- ✓ La instrucción `with` queda prohibida

3.7 COMPROBACIÓN DE TIPOS DE ARGUMENTOS

Comprobar el tipo real en tiempo de ejecución

Como no se declaran tipos, una función puede aceptar distintos tipos y decidir qué hacer según el tipo real del argumento.

Tipo	Prueba
Cadena	<code>typeof x === 'string'</code>
Expresión regular	<code>x instanceof RegExp</code>
Número	<code>typeof x === 'number'</code>
Convertible a número	<code>typeof +x === 'number'</code>
Vector (array)	<code>Array.isArray(x)</code>
Función	<code>typeof x === 'function'</code>

3.8 MÁS O MENOS ARGUMENTOS

JavaScript nunca exige el número exacto

```
const promedio = (x, y) => (x + y) / 2
promedio(3, 4, 5)
// 3.5 — se ignora el último argumento
```

⚠️ Argumentos de más → se ignoran en silencio

```
promedio(3)
// NaN — (3 + undefined) / 2

const promedio = (x, y) =>
  y === undefined ? x : (x + y) / 2
```

⚠️ Argumentos de menos → se establecen como undefined

3.9 ARGUMENTOS PREDETERMINADOS

Un valor por defecto en la declaración

```
const promedio = (x, y = x) => (x + y) / 2

const promedio = (x = 0, y = x) => (x + y) / 2
// promedio() ahora devuelve 0
```

Llamada	Equivale a
promedio(3)	promedio(3, undefined)
promedio()	promedio(0, undefined)
promedio(undefined, 3)	promedio(0, 3)

3.10 PARÁMETROS RESTANTES Y PROPAGACIÓN

rest y spread: acciones opuestas

SPREAD — EXPANDE UN VECTOR EN ARGUMENTOS

`numeros = [1, 7, 2, 9]`



`Math.max(...numeros)`



9

REST — RECOGE LOS ARGUMENTOS EN UN VECTOR

`suma(1, 7, 2, 9)`



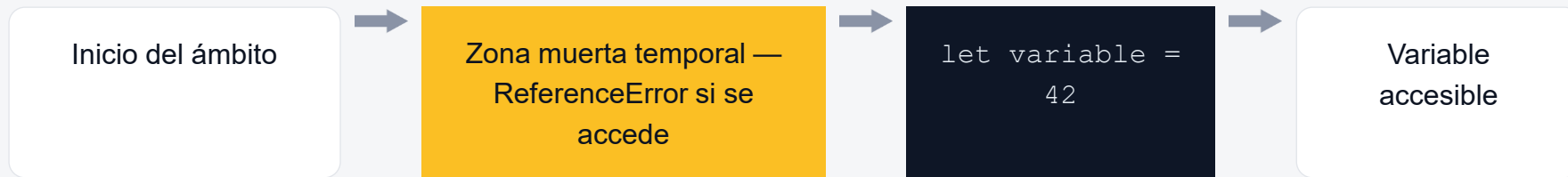
`primero = 1, siguientes = [7, 2, 9]`

```
const suma = (primero = 0, ...siguientes) => {  
  let total = primero  
  for (const valor of siguientes) { total += valor }  
  return total  
}
```

3.11 HOISTING

Toda declaración se eleva a su ámbito

JavaScript reserva espacio para variables y funciones antes de ejecutar el código, aunque se declaren más abajo.



```
function esPar(n) { return n === 0 ? true : esImpar(n - 1) }  
function esImpar(n) { return n === 0 ? false : esPar(n - 1) }  
// gracias al hoisting, pueden ser mutuamente recursivas
```

3.12 LANZAMIENTO DE EXCEPCIONES

throw: la instrucción para lanzar una excepción

Si una función no puede calcular un resultado, puede **lanzar una excepción** en vez de devolver un valor de error.

⚠ La función se termina de inmediato: no hay valor de retorno, ni siquiera undefined.

💡 Úsalas para lo imprevisible, no para validar entradas de usuario que ya esperas que fallen.

```
let razon = `Elemento ${elem} no encontrado`  
throw Error(razon)
```

3.13 CAPTURA DE EXCEPCIONES

try / catch: gestionar el fallo

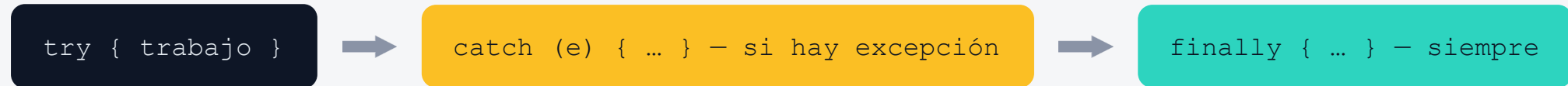
```
try {  
    // realizar trabajo  
} catch (e) {  
    console.log(e)  
    throw e  
    // reenviar al controlador  
}
```

La variable de **catch** (aquí, **e**) contiene el valor de la excepción: un objeto de error con las propiedades **name** y **message**.

⚠ A diferencia de Java o C++, una cláusula catch captura **todas** las excepciones, sin distinguir por tipo.

3.14 LA CLÁUSULA FINALLY

Se ejecuta siempre, pase lo que pase



Con éxito, con return/break, o con una excepción: la cláusula finally se ejecuta a continuación en todos los casos.

```
try {  
    // adquirir recursos y realizar el trabajo  
} finally {  
    // liberar recursos, ocurra lo que ocurra  
}
```

RESUMEN

Ideas clave del tema



Funciones = valores

Son de primera clase: se guardan, se pasan y se devuelven como cualquier otro dato.



function vs =>

Los literales y las funciones flecha son la forma habitual de escribir funciones cortas.



map, filter, forEach

Procesan matrices sin bucles explícitos, encadenando transformaciones.



Argumentos flexibles

Valores predeterminados, rest y spread dan control sobre cuántos argumentos llegan.



Hoisting

Declara antes de usar y evita var: así el hoisting nunca sorprende.



try / catch / finally

throw lanza, catch gestiona y finally libera recursos siempre.